

GUÍA DE DESARROLLO E IMPLEMENTACIÓN

Evaluación Parcial N.º 1 (EP1)

Microsoft Entra ID + React + Vite + TypeScript + MSAL + AWS API Gateway + BFF

Docente: Patricio Silva Gutiérrez.

Adaptación técnica de la guía: el frontend de este caso utiliza React 19, Vite, TypeScript, React Router, @azure/msal-react y el cliente HTTP src/api/client.ts. La autenticación, las rutas protegidas y los claims se demuestran con RequireAuth y RequireCatalogRole.

Propósito: esta versión define con precisión el mínimo funcional y técnico que debe entregar cada grupo en la EP1

1. Contexto y objetivo de la EP1

Su objetivo es demostrar una integración segura y funcional entre identidad, frontend, API Gateway, BFF y los microservicios mínimos del dominio.

La plataforma de identidad utilizada en esta guía es Microsoft Entra ID estándar. Por ello, los usuarios de prueba deben existir previamente en el tenant. No se exige implementar Microsoft Entra External ID ni User Flows de autoregistro.

Principio de esta versión: el mínimo funcional no significa reducir cada caso a un único endpoint. Cada caso debe conservar los módulos mínimos necesarios para diferenciar sus actores y demostrar autenticación y autorización de forma real.

2. Qué debe demostrar la EP1

Indicador

React + Vite + TypeScript + MSAL

Ponderación

60%

BFF + JWT

40%

Evidencia esperada

Login, logout, protección de rutas,

Access Token, cliente HTTP React (api/client.ts) y

consumo seguro del API Gateway.

Validación de issuer, audience, firma y

vigencia del JWT; autorización por

roles cuando corresponda.

3. Arquitectura mínima obligatoria

Regla arquitectónica: el BFF no se conecta directamente a la base de datos. Su responsabilidad es validar y autorizar la

solicitud y coordinar las llamadas hacia los microservicios del dominio.

4. Alcance obligatorio común a todos los casos

- [illegible]

Microsoft Entra ID: tenant, usuarios de prueba y configuración de roles.

App Registration del frontend React.

App Registration de la API o configuración equivalente del recurso protegido.

Redirect URI, Client ID, Tenant ID, Authority y permisos/scopes requeridos.

React integrado con MSAL.

Login y logout.

Rutas protegidas con RequireAuth y RequireCatalogRole.

cliente HTTP React (api/client.ts) adjuntando Authorization: Bearer <access_token>.

AWS API Gateway como único punto público de entrada al backend.

Validación JWT en API Gateway.

CORS y ruta OPTIONS para el frontend.

BFF con Spring Security.

Segunda validación del JWT en el BFF.

Microservicio principal del caso.

Microservicio de catálogo del caso.

Persistencia mínima para demostrar la funcionalidad.

Pantallas mínimas diferenciadas según los actores del caso.

5. Paso a paso técnico común

Paso 1. Microsoft Entra ID

Crear o seleccionar el tenant y disponer de usuarios de prueba.

Paso 2. Roles

Crear y asignar los roles del caso que permitan diferenciar las acciones de los actores.

Paso 3. Frontend

Registrar React como SPA y configurar Client ID, Tenant ID, Authority y Redirect URI.

Paso 4. API

Registrar/configurar el recurso protegido, sus scopes y la audiencia esperada.

Paso 5. MSAL

Implementar login, logout, RequireAuth y RequireCatalogRole y cliente HTTP React (api/client.ts).

Paso 6. API Gateway

Crear todas las rutas funcionales mínimas del caso y la ruta OPTIONS; configurar JWT Authorizer y CORS.

Paso 7. BFF

Implementar Spring Security y validar issuer, audience, firma y vigencia.

Paso 8. Microservicios

Implementar el microservicio principal y el microservicio de catálogo indicados para el caso.

Paso 9. Frontend funcional

Implementar las pantallas mínimas del caso y mostrar diferencias reales según el rol.

Paso 10. Pruebas

Mostrar respuestas correctas y rechazo de solicitudes sin token, con token inválido o sin permisos.

7. Reglas comunes de autorización

Componente

Microsoft Entra ID

React + Vite + TypeScript + MSAL

RequireAuth / RequireCatalogRole

cliente HTTP React (api/client.ts)

API Gateway

BFF

Microservicios

Responsabilidad

Autentica usuarios y emite los tokens; mantiene usuarios y roles configurados.

Inicia/cierra sesión y obtiene tokens.

Evita el acceso a rutas que requieren autenticación.

Adjunta el Access Token a las llamadas hacia el recurso protegido.

Valida el JWT y rechaza solicitudes que no cumplen la política.

Valida nuevamente el JWT y aplica autorización por rol cuando corresponde.

Ejecutan la lógica de negocio y persisten datos.

8. Casos 0 al 6: mínimo obligatorio para la EP1

Cada caso conserva su módulo principal y su catálogo, porque esa combinación permite diferenciar actores, permisos y reglas de negocio.

Caso 0 — Pedidos360

Actores

Actor

Admin

Operador

Cliente

Responsabilidad

Administra productos, stock y la operación global.

Gestiona pedidos, cambios de estado y operación diaria.

Crea y sigue sus propios pedidos.

Alcance funcional mínimo

Módulo

Gestión de pedidos

Descripción

CRUD de pedidos y cambio

de estado: CREADO

ACEPTADO

EN_PREPARACIÓN

DESPACHADO

ENTREGADO / CANCELADO.

CRUD de productos, precios
y stock disponible.

Catálogo

Actores

Cliente, Operador, Admin

Regla clave

No se puede DESPACHAR sin
ACEPTAR.

Admin, Operador

El stock disminuye al
aceptar el pedido.

Microservicios mínimos

Servicio

ms-pedidos360orders

Dominio

Pedidos

DB

Oracle

ms-pedidos360catalog

Productos / Stock

Oracle

Responsabilidad

CRUD de pedidos,

estados y

coordinación de

stock.

CRUD de productos,

stock y precios.

Exposición

/api/orders/*

/api/catalog/*

Pantallas mínimas

Ruta

/login

Roles

Público

Componente

LoginPage

/dashboard

/orders

Admin, Operador, Cliente

Admin, Operador, Cliente

DashboardPage

OrdersPage

/catalog

Admin, Operador

CatalogPage

Función

MSAL. Botón de inicio de sesión.

Resumen distinto por rol.

Cliente: sus pedidos;

Operador/Admin: gestión y estados.

Productos, precios y control de stock.

Diferenciación mínima entre actores

Acción

Ver catálogo

Crear pedido

Ver todos los pedidos

Cambiar estado

Crear/editar producto

Administrar stock

Admin

Sí

Sí

No

No

No

No

Operador

Sí

Sí

Sí

Sí

Según operación

Sí

Cliente

Sí

Sí

Sí

Sí

Sí

Sí

Caso 1 — VidaSalud

Actores

Actor

Admin

Responsabilidad

Coordina la red, el catálogo de prestaciones y la visión global.

Confirma atenciones y gestiona el flujo operativo.

Solicita y sigue sus atenciones.

Recepcionista

Paciente

Alcance funcional mínimo

Módulo

Gestión de atenciones

Descripción

CRUD de atenciones y cambio de estado:

SOLICITADA

CONFIRMADA

EN_ESPERA

EN_ATENCIÓN CERRADA

/ CANCELADA.

CRUD de prestaciones, boxes y cupos disponibles.

Catálogo

Actores

Paciente, Recepcionista,

Admin

Regla clave

No se puede pasar a

EN_ATENCIÓN sin

CONFIRMAR.

Admin, Recepcionista

El cupo del box disminuye al confirmar la atención.

Microservicios mínimos

Servicio

ms-vidasaludappointments

ms-vidasalud-catalog

Dominio

Atenciones

DB

Oracle

Prestaciones / Boxes /

Cupos

Oracle

Responsabilidad

CRUD de atenciones y

cambios de estado.

CRUD de

prestaciones, boxes y

cupos.

Exposición

/api/appointments/*

/api/catalog/*

Pantallas mínimas

Ruta

/login

/dashboard

Roles

Público

Admin, Recepcionista,

Paciente

Componente

LoginPage

DashboardPage

/appointments

Admin, Recepcionista,

Paciente

AppointmentsPage

/catalog

Admin, Recepcionista

CatalogPage

Función

MSAL.

Admin: KPIs; Recepcionista:

pendientes; Paciente:

próximas atenciones.

Paciente crea/sigue;

Recepcionista/Admin

gestiona estados.

Prestaciones, boxes y cupos.

Recepcionista

Sí

Sí

Sí

Sí

Según operación

Paciente

Sí

Sí

Sí

Sí

Sí

Diferenciación mínima entre actores

Acción

Crear atención

Ver propias atenciones

Ver todas las atenciones

Cambiar estado

Gestionar catálogo

Admin

Sí

Sí

No

No

No

Caso 2 — CampusLab

Actores

Actor

Admin

Técnico

Responsabilidad

Administra laboratorios, equipos e insumos.

Aprueba reservas, prepara salas y registra devoluciones.

Estudiante

Solicita y sigue sus reservas.

Alcance funcional mínimo

Módulo

Gestión de reservas

Descripción

CRUD de reservas y cambio

de estado: SOLICITADA

APROBADA

EN_PREPARACIÓN

EN_USO DEVUELTA /

CANCELADA.

CRUD de laboratorios,

equipos e insumos

disponibles.

Catálogo

Actores

Estudiante, Técnico, Admin

Regla clave

No se puede pasar a

EN_USO sin APROBAR.

Admin, Técnico

El stock o cupo disminuye al

aprobar la reserva.

Microservicios mínimos

Servicio

ms-campuslabbookings

ms-campuslab-catalog

Dominio

Reservas

DB

Oracle

Laboratorios /

Equipos / Insumos

Oracle

Responsabilidad
CRUD de reservas y
cambios de estado.
CRUD y actualización
de cupo o stock.

Exposición
/api/bookings/*
/api/catalog/*

Pantallas mínimas

Ruta
/login
/dashboard

Roles
Público
Admin, Técnico, Estudiante

Componente
LoginPage
DashboardPage

/bookings
Admin, Técnico, Estudiante
BookingsPage

/catalog
Admin, Técnico

CatalogPage

Función
MSAL.
Ocupación, preparación y
próximas reservas según rol.
Estudiante solicita;
Técnico/Admin aprueba y
cambia estado.
Laboratorios, equipos e
insumos.

Diferenciación mínima entre actores

Acción
Crear reserva

Ver propias reservas
Ver todas las reservas
Cambiar estado
Gestionar catálogo

Admin

Sí

Sí

No

No

No

Técnico

Sí

Sí

Sí

Sí

Según operación

Estudiante

Sí

Sí

Sí

Sí

Sí

Caso 3 — RutaExpress

Actores

Actor

Admin

Despachador

Cliente

Responsabilidad

Administra servicios y capacidad.

Acepta envíos, arma rutas y marca entregas.

Crea y sigue sus envíos.

Alcance funcional mínimo

Módulo

Gestión de envíos

Descripción

CRUD de envíos y cambio de

estado: CREADO
ACEPTADO EN_BODEGA
EN_RUTA ENTREGADO
/ CANCELADO.

Actores

Cliente, Despachador,
Admin

Regla clave

No se puede pasar a
EN_RUTA sin ACEPTAR.

Catálogo

CRUD de servicios de envío y
capacidad disponible.

Admin, Despachador

La capacidad disminuye al
aceptar el envío.

Microservicios mínimos

Servicio

ms-rutaexpressshipments

ms-rutaexpresscatalog

Dominio

Envíos

DB

Oracle

Servicios / Capacidad

Oracle

Responsabilidad

CRUD de envíos y
cambios de estado.

CRUD de servicios,
tarifas y capacidad.

Exposición

/api/shipments/*

/api/catalog/*

Pantallas mínimas

Ruta

/login

/dashboard

/shipments

/catalog

Roles

Público

Admin, Despachador,

Cliente

Admin, Despachador,

Cliente

Componente

LoginPage

DashboardPage

Admin, Despachador

CatalogPage

ShipmentsPage

Función

MSAL.

KPIs, operación y
seguimiento según rol.

Cliente crea/sigue;

Despachador/Admin

gestiona estados.

Tipos de envío, tarifas y
capacidad.

Diferenciación mínima entre actores

Acción

Crear envío

Ver propios envíos

Ver todos los envíos

Cambiar estado

Gestionar catálogo

Admin

Sí

Sí

No

No

No

Despachador

Sí

Sí

Sí

Sí

Según operación

Cliente

Sí

Sí
Sí
Sí
Sí

Caso 4 — BarrioDigital

Actores

Actor

Admin

Funcionario

Vecino

Responsabilidad

Define tipos de trámite y cupos.

Admite solicitudes y gestiona el trámite.

Ingresa y sigue sus solicitudes.

Alcance funcional mínimo

Módulo

Gestión de trámites

Descripción

CRUD de trámites y cambio

de estado: INGRESADO

ADMITIDO EN_GESTIÓN

EN_TERRENO

RESUELTO / RECHAZADO.

CRUD de tipos de trámite y

cupos diarios disponibles.

Catálogo

Actores

Vecino, Funcionario, Admin

Regla clave

No se puede pasar a

EN_TERRENO sin ADMITIR.

Admin, Funcionario

El cupo diario disminuye al

admitir el trámite.

Microservicios mínimos

Servicio

ms-barriodigitalrequests

ms-barriodigitalcatalog

Dominio

Trámites

DB

Oracle

Tipos de trámite /

Cupos

Oracle

Responsabilidad

CRUD de solicitudes y
cambios de estado.

CRUD de
procedimientos,
requisitos y cupos.

Exposición

/api/requests/*

/api/catalog/*

Pantallas mínimas

Ruta

/login

/dashboard

Roles

Público

Admin, Funcionario, Vecino

Componente

LoginPage

DashboardPage

/requests

Admin, Funcionario, Vecino

RequestsPage

/catalog

Admin, Funcionario

CatalogPage

Función

MSAL.

KPIs, cola operativa y
seguimiento según rol.

Vecino crea/sigue;

Funcionario/Admin cambia
estados.

Tipos de solicitud y cupos
diarios.

Diferenciación mínima entre actores

Acción

Crear trámite

Ver propios trámites

Ver todos los trámites

Cambiar estado

Gestionar catálogo

Admin

Sí

Sí

No

No

No

Funcionario

Sí

Sí

Sí

Sí

Según operación

Vecino

Sí

Sí

Sí

Sí

Sí

Caso 5 — AndesStay

Actores

Actor

Admin

Recepcionista

Huésped

Responsabilidad

Administra el inventario de unidades.

Confirma reservas y realiza check-in/check-out.

Crea y sigue sus reservas.

Alcance funcional mínimo

Módulo

Gestión de reservas

Descripción

CRUD de reservas y cambio

de estado: CREADA

CONFIRMADA

CHECKIN_PENDIENTE

EN_ESTADÍA CHECKOUT /

CANCELADA.

CRUD de unidades

(habitación/cabaña) y

disponibilidad.

Catálogo

Actores

Huésped, Recepcionista,
Admin

Regla clave

No se puede hacer check-in
sin CONFIRMAR.

Admin, Recepcionista

La disponibilidad disminuye
al confirmar la reserva.

Microservicios mínimos

Servicio

ms-andesstayreservations

ms-andesstay-catalog

Dominio

Reservas

DB

Oracle

Unidades /

Disponibilidad

Oracle

Responsabilidad

CRUD de reservas y
cambios de estado.

CRUD de unidades,
tarifas y
disponibilidad.

Exposición

/api/reservations/*

/api/catalog/*

Pantallas mínimas

Ruta

/login

/dashboard

/reservations

Roles

Público

Admin, Recepcionista,

Huésped

Admin, Recepcionista,

Huésped

Componente

LoginPage

DashboardPage

Función

MSAL.

Ocupación, llegadas/salidas

y reservas según rol.

ReservationsPage

Huésped crea/sigue;

Recepcionista/Admin

cambia estado.

/catalog

Admin, Recepcionista

CatalogPage

Habitaciones, cabañas,
tarifas y disponibilidad.

Recepcionista

Sí

Sí

Sí

Sí

Según operación

Huésped

Sí

Sí

Sí

Sí

Sí

Diferenciación mínima entre actores

Acción

Crear reserva

Ver propias reservas

Ver todas las reservas

Cambiar estado

Gestionar catálogo

Admin

Sí

Sí

No

No

No

Caso 6 — DigitalFix

Actores

Actor

Admin

Supervisor

Cliente

Responsabilidad

Administra servicios técnicos y repuestos.

Acepta órdenes, asigna técnicos y cierra trabajos.

Crea y sigue sus órdenes de mantenimiento.

Alcance funcional mínimo

Módulo

Gestión de órdenes

Descripción

CRUD de órdenes y cambio

de estado: CREADA

ASIGNADA

EN_DESPLAZAMIENTO

EN_EJECUCIÓN CERRADA

/ CANCELADA.

CRUD de servicios técnicos y

stock de repuestos.

Catálogo

Actores

Cliente, Supervisor, Admin

Regla clave

No se puede pasar a

EN_EJECUCIÓN sin

ASIGNAR.

Admin, Supervisor

El stock del repuesto

disminuye al asignar la

orden.

Microservicios mínimos

Servicio

ms-digitalfixworkorders

ms-digitalfix-catalog

Dominio

Órdenes de trabajo

DB

Oracle

Servicios / Repuestos

Oracle

Responsabilidad

CRUD de órdenes y
cambios de estado.

CRUD de servicios,
tarifas y stock.

Exposición

/api/workorders/*

/api/catalog/*

Pantallas mínimas

Ruta

/login

/dashboard

Roles

Público

Admin, Supervisor, Cliente

Componente

LoginPage

DashboardPage

/workorders

Admin, Supervisor, Cliente

WorkordersPage

/catalog

Admin, Supervisor

CatalogPage

Función

MSAL.

KPIs, órdenes pendientes y
seguimiento según rol.

Cliente crea/sigue;

Supervisor/Admin cambia
estado.

Servicios, repuestos y stock.

Diferenciación mínima entre actores

Acción

Crear orden
Ver propias órdenes
Ver todas las órdenes
Cambiar estado
Gestionar catálogo

Admin
Supervisor
Cliente

Sí
Sí
Sí
Sí
Sí
Sí
No
Sí
Sí
No
Sí
Sí
No
Según operación
Sí

9. Mínimo técnico por caso

Cada caso debe entregar, como mínimo, los siguientes repositorios o proyectos:

Proyecto

frontend-<caso>

Contenido mínimo

React + Vite + TypeScript + MSAL + login/logout + RequireAuth y RequireCatalogRole + cliente HTTP React (api/client.ts) + pantallas mínimas.

Spring Boot + Spring Security + validación JWT + autorización + llamadas a microservicios.

Spring Boot + lógica del módulo principal + persistencia.

Spring Boot + lógica de catálogo + persistencia.

ms-<caso>-bff

ms-<caso>-principal

ms-<caso>-catalog

10. Flujo completo que se debe demostrar

-
-
-
-
-
-
-
-
-
-
-
-
-
-
-

1. Usuario abre la aplicación React.

2. Usuario selecciona Login.

3. MSAL redirige a Microsoft Entra ID.

4. Usuario inicia sesión con un usuario de prueba.

5. MSAL procesa la sesión y obtiene el Access Token.

6. React muestra pantallas o acciones según el rol.
7. El usuario navega a una ruta protegida.
8. React invoca una ruta funcional a través de API Gateway.
9. cliente HTTP React (api/client.ts) adjunta Authorization: Bearer <access_token>.
10. API Gateway valida el JWT y enruta al BFF.
11. El BFF valida nuevamente issuer, audience, firma y vigencia.
12. El BFF aplica autorización por rol cuando corresponde.
13. El BFF llama al microservicio principal o de catálogo.
14. El microservicio ejecuta la regla de negocio y persiste/consulta datos.
15. La respuesta JSON vuelve a React.
16. La interfaz muestra el resultado correspondiente al actor autenticado.

11. Pruebas mínimas obligatorias

Prueba

Sin token

Token inválido o expirado

Token válido con rol autorizado

Token válido sin permiso suficiente

Regla de negocio inválida

Regla de negocio válida

Resultado esperado

La solicitud protegida es rechazada.

La solicitud es rechazada.

La solicitud es procesada correctamente.

La solicitud es rechazada por autorización.

La operación no se ejecuta.

La operación se completa y devuelve el JSON esperado.

12. Evidencias mínimas de entrega

-
-
-
-
-
-

-

Tenant de Microsoft Entra ID y usuarios de prueba.

Roles configurados para los actores del caso.

App Registration del frontend.

Configuración de Redirect URI, Client ID y Tenant ID.

Configuración de scopes/audience del recurso protegido.

Login y logout funcionando.

Rutas protegidas con RequireAuth y RequireCatalogRole.

-
-
-
-
-
-
-
-
-
-
-

cliente HTTP React (api/client.ts) agregando el Access Token.

Pantallas diferenciadas por actor.

Rutas funcionales creadas en API Gateway.

JWT Authorizer configurado.

CORS y OPTIONS configurados.

BFF desplegado y validando JWT.

Microservicio principal desplegado.

Microservicio de catálogo desplegado.

Pruebas de autorización y reglas de negocio.

Respuesta visible desde React.

Código fuente organizado en repositorios separados.

13. Checklist final

- [] Tengo usuarios de prueba para los roles mínimos del caso.
- [] Los roles generan diferencias reales en la aplicación.
- [] Login funciona con Microsoft Entra ID.
- [] Logout funciona.
- [] Las rutas privadas están protegidas.
- [] React obtiene y envía un Access Token.
- [] API Gateway rechaza solicitudes no autorizadas.
- [] API Gateway enruta todas las rutas mínimas del caso.
- [] El BFF valida el JWT nuevamente.
- [] El BFF no se conecta directamente a la base de datos.
- [] Existe el microservicio principal.
- [] Existe el microservicio de catálogo.
- [] Las reglas de negocio mínimas funcionan.
- [] Las pantallas permiten demostrar diferencias entre actores.
- [] El flujo completo funciona de extremo a extremo.

14. Resumen final para el estudiante

Debes implementar correctamente el mínimo funcional y técnico definido para tu caso.

MICROSOFT ENTRA ID

Usuarios + Roles

REACT + VITE + TYPESCRIPT + MSAL

Pantallas diferenciadas por actor

AWS API GATEWAY

JWT + CORS + Rutas

BFF

Validación JWT + Autorización

MICROSERVICIO PRINCIPAL

+

MICROSERVICIO DE CATÁLOGO

REGLAS DE NEGOCIO Y DATOS

RESPUESTA VISIBLE EN REACT

Objetivo principal: demostrar de extremo a extremo autenticación, autorización, validación JWT, integración entre

componentes y diferencias reales entre los actores del caso.

Anexo de identidad dual y despliegue AWS

Pedidos360 · React 19 + Vite + TypeScript + MSAL + Amazon Cognito

Este anexo actualiza la guía EP1 para la implementación entregada con dos proveedores de identidad coexistentes. La separación es deliberada: Entra ID protege **/api/**** y Amazon Cognito protege **/aws/api/****. Los dos accesos no comparten token, audience, callback ni estado de autorización.

Namespace	Proveedor	Callback SPA	Decoder backend
/api/**	Microsoft Entra ID	/login	Issuer/audience Entra; roles
/aws/api/**	Amazon Cognito	/auth/cognito/callback	Issuer Cognito; audience/client_id; token_use=access; cognito:groups

1. Flujo de autenticación del frontend

React mantiene MSAL y Cognito en contextos separados. El botón de Microsoft adquiere un access token para el scope de Entra y el cliente HTTP lo envía a **/api/****. El botón de Amazon Cognito usa Authorization Code + PKCE S256, sin client secret, y el portal **/aws** usa exclusivamente **/aws/api/****. El callback Cognito es una ruta independiente para que MSAL no procese el código OAuth de Cognito.

```
Entra: /login → MSAL → /api/**  
Cognito: /auth/cognito/callback → Amplify → /aws/api/**
```

El SDK de Cognito se configura con `sessionStorage` y fallback de memoria. No se guardan contraseñas, client secrets, AWS access keys ni tokens en el código, el bundle, Terraform o logs. El backend vuelve a validar criptográficamente cada token.

2. Configuración del App Client de Cognito

Valores públicos utilizados por la implementación:

```
User Pool: us-east-1_UmEhPRyDI  
App Client ID: 59be26pgg5ginu2sutr8eetggjg  
Hosted UI: us-east-1umehprydi.auth.us-east-1.amazoncognito.com  
Issuer: https://cognito-idp.us-east-1.amazonaws.com/us-east-1_UmEhPRyDI  
JWKS: https://cognito-idp.us-east-1.amazonaws.com/us-east-1_UmEhPRyDI/.well-known/jwks.json
```

Configuración	Valor requerido
Allowed callback URL local	http://localhost:4200/auth/cognito/callback
Allowed callback URL AWS	https://<FRONTEND_ORIGIN>/auth/cognito/callback
Allowed sign-out URL	http://localhost:4200/login y https://<FRONTEND_ORIGIN>/login
Grant type	Authorization Code únicamente
PKCE	S256 obligatorio
Client secret	Desactivado; la SPA es cliente público
Scopes	openid, email, profile
Grupos	Admin, Operador, Cliente; mayúsculas exactas

Importante: ``redirect_mismatch`` significa que la URL que Cognito está rechazando no coincide exactamente con una URL registrada. No registrar una barra final, query string o ruta distinta a la documentada.

3. Validación de JWT y autorización

Cada backend crea dos cadenas Spring Security y dos decoders. El decoder de Cognito exige issuer y JWKS exactos, audience o `client\_id` baseline, `token\_use=access` y el claim `cognito:groups`. El frontend solo envía access tokens; un ID token nunca se acepta como Bearer de API.

Token	/api/**	/aws/api/**
Entra access válido	200/403 según rol	401
Cognito access válido	401	200/403 según grupo
Cognito ID token	401	401
Token expirado / otra audience	401	401
Autenticado sin grupo reconocido	403	403

Rol y responsabilidad

Rol	Permisos de negocio
Admin	Gestiona catálogo, stock, pedidos y transiciones.
Operador	Gestiona catálogo, stock, pedidos y transiciones.
Cliente	Crea y consulta sus propios pedidos; cancela los propios en CREADO.

Los claims solo-controlan la interfaz; la autorización real se aplica en BFF, Catalog y Orders. Un usuario sin uno de los tres grupos no se convierte implícitamente en Cliente.

4. API Gateway y propagación privada

API Gateway HTTP API es la única entrada pública del backend. Terraform crea un authorizer JWT de Entra para las rutas `/api/\*\*`, uno de Cognito para `/aws/api/\*\*`, CORS con orígenes exactos, VPC Link y las doce rutas públicas por proveedor. Las rutas Cognito exigen el scope `openid`; no hay ruta `\$default` y las reservas internas no se publican.

Navegador

```
■■ Entra access token ■■> /api/** ■■> authorizer Entra ■■> BFF privado
■■ Cognito access token ■> /aws/api/** ■> authorizer Cognito ■> BFF privado
■■> /aws/api/** en Catalog/Orders
```

El BFF conserva el prefijo del proveedor al reenviar a los microservicios. Orders elige el namespace de Catalog según la petición de entrada y propaga el mismo Bearer ya validado. BFF, Catalog, Orders y Oracle permanecen sin ingress público.

5. GitHub Actions, ECR y Terraform

Cada repositorio de aplicación tiene un workflow manual `publish-ecr.yml`. El workflow usa GitHub OIDC, no access keys. Terraform crea cuatro repositorios ECR con cifrado AES256 y scan-on-push para frontend, BFF, Catalog y Orders.

Componente	Flujo de entrega
Frontend	npm test/build → imagen Nginx en ECR → hosting estático público.
BFF/Catalog/Orders	mvn verify → imagen JRE no root en ECR → ECS/Fargate privado.
Infraestructura	terraform fmt/validate/plan → aprobación protegida → apply.
Identidad AWS	Rol de GitHub OIDC con permisos mínimos a ECR y plan/apply.

Variables públicas de build

```
API_BASE_URL=https://<API_GATEWAY>
REDIRECT_URI=https://<FRONTEND>/login
COGNITO_REDIRECT_URI=https://<FRONTEND>/auth/cognito/callback
COGNITO_LOGOUT_URI=https://<FRONTEND>/login
COGNITO_USER_POOL_ID / CLIENT_ID / DOMAIN / ISSUER
```

Las variables son identificadores públicos. No guardar `client\_secret`, contraseñas Oracle, connection strings, AWS keys, refresh tokens ni archivos `\*.tfvars` en GitHub. Oracle y secretos se inyectan en runtime mediante el mecanismo privado adoptado por la infraestructura.

6. Checklist de entrega

- El App Client Cognito tiene Authorization Code, PKCE S256, sin client secret y callbacks exactos.
- Los grupos Admin, Operador y Cliente existen con mayúsculas exactas y están asignados a usuarios de prueba.
- El frontend React presenta ambos accesos y procesa callbacks separados.
- Un token Entra en /aws/api y un token Cognito en /api producen 401.
- Un Cognito ID token produce 401; solo se acepta access token.
- Los servicios niegan usuarios sin grupo reconocido con 403.
- API Gateway tiene dos authorizers, doce rutas por namespace y ningún catch-all.
- Las cuatro imágenes se publican en ECR mediante OIDC y se registran por digest.
- BFF, Catalog, Orders y Oracle no tienen URL pública; API Gateway es la única entrada del backend.
- La evidencia no contiene JWT completos, secretos ni datos personales innecesarios.

La configuración completa y los comandos de validación están en `docs/CONFIGURACION\_COGNITO.md`, `docs/DESPLIEGUE\_AWS.md` y `docs/ENTREGA\_Y\_EVIDENCIAS.md` del repositorio de infraestructura.